

Assignment 1: Perceptron and ANN BP for Classification

Objectives:

- To understand the implementation of Perceptron for the classification of points belonging to two linear separable classes using python.
- To know how to implement ANN using the **Keras** framework for general classification tasks.
- To know how to display nice graphics using python.
- To know how to implement ANN using the **MLPClassifier** framework.

Given file: Class02C_Assignment1_on_Classification.ipynb

ELEC 6604 Assignment 1

```
import numpy as np
import matplotlib.pyplot as plt
from sklearn.datasets import make_moons
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.neural_network import MLPClassifier
from sklearn.metrics import accuracy_score
import copy
```

```
# ANN Training
```

```
import pandas as pd
import random
import time
import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, BatchNormalization
from tensorflow.keras.layers import Dropout
from tensorflow.keras.optimizers import Adam, SGD
```

Define/import the
libraries

```

# Part 1: Perceptron 12 data points Linearly Separable
#-----
# Perceptron parameters
W12=-3
W13= 2
theta = -1
learnrate = 0.3

# training instances are defined
tmpp= np.array([
    [1.0000 , 1.0000 , 1.0000],
    [1.0000 , 0.5000 , 1.0000],
    [1.5000 , 0.2000 , 1.0000],
    [0.5000 , 0.7500 , 1.0000],
    [1.2000 , 1.5000 , 1.0000],
    [1.6000 , 1.2000 , 1.0000],
    [0.5000 , 0.3000 , 0.0000],
    [0.5000 , -0.7000 , 0.0000],
    [0.0000 , 0.2 , 0.0000],
    [-0.7 , 0.9000 , 0.0000],
    [-0.6 , -0.5 , 0.0000],
    [-0.5 , 0.5 , 0.0000]
])
print(tmpp.shape)

```

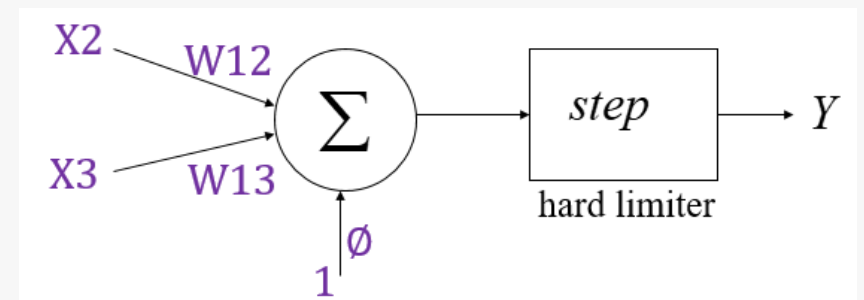
Perceptron (two I/P, one O/P):
Define the parameters
and 12 data points

(12, 3)

```

X2=tmp[(:,0)]
X3=tmp[(:,1)]
desiredout=tmp[(:,2)]
print('X2 = ', X2)
print('X3 = ', X3)
print('desiredout = ',desiredout)

```



```

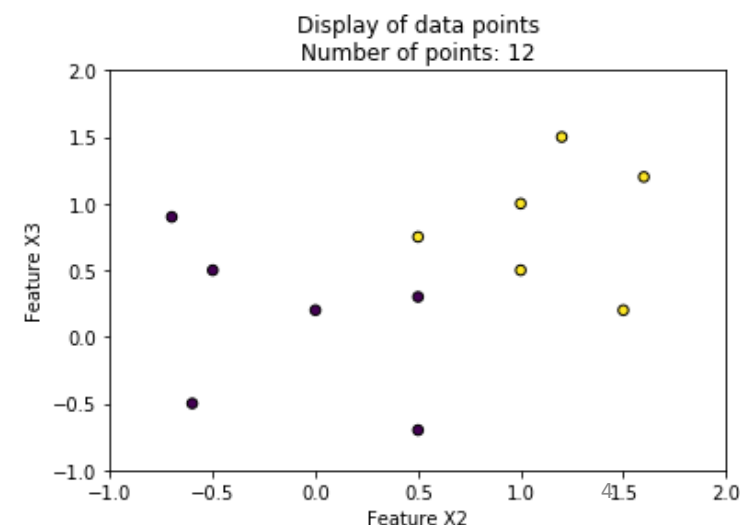
X2 = [ 1.    1.    1.5  0.5  1.2  1.6  0.5  0.5  0.  -0.7 -0.6 -0.5]
X3 = [ 1.    0.5  0.2  0.75 1.5  1.2  0.3  -0.7  0.2  0.9  -0.5  0.5 ]
desiredout = [1. 1. 1. 1. 1. 1. 0. 0. 0. 0. 0. 0.]

```

```

# plot
Num_datapoints = len(X2)
plt.scatter(X2, X3, c=desiredout, cmap='viridis', edgecolors='k', s=30)
plt.title(f'Display of data points\n'
          f'Number of points: {Num_datapoints}')
plt.xlabel('Feature X2')
plt.ylabel('Feature X3')
plt.xlim(-1, 2) # xmin=-1, xmax=2
plt.ylim(-1, 2)
plt.show()

```

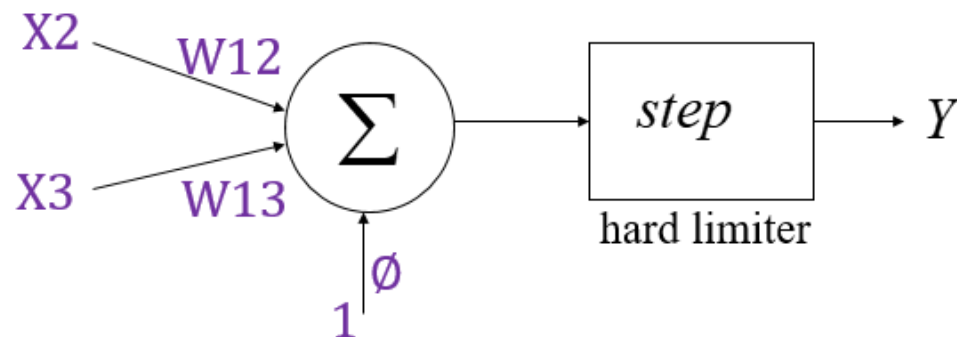


```

# training of perceptrons
out = -1
for iteration in range(10): # i=0,1,2,3,...,9
    totalerror=0
    for j in range(Num_datapoints): # j=0,1,2,3,...
        tmpout=W12*X2[j]+W13*X3[j]+theta
        if (tmpout > 0):
            out = 1
        else:
            out = 0
        diff = desiredout[j] - out
        W12 = W12 + learnrate*diff*X2[j]; # error correction learning rule
        W13 = W13 + learnrate*diff*X3[j];
        theta = theta + learnrate*diff;
        totalerror = totalerror + np.abs(diff);
        #debug print(iteration, j, totalerror, W12, W13, theta)

```

diff is used to update



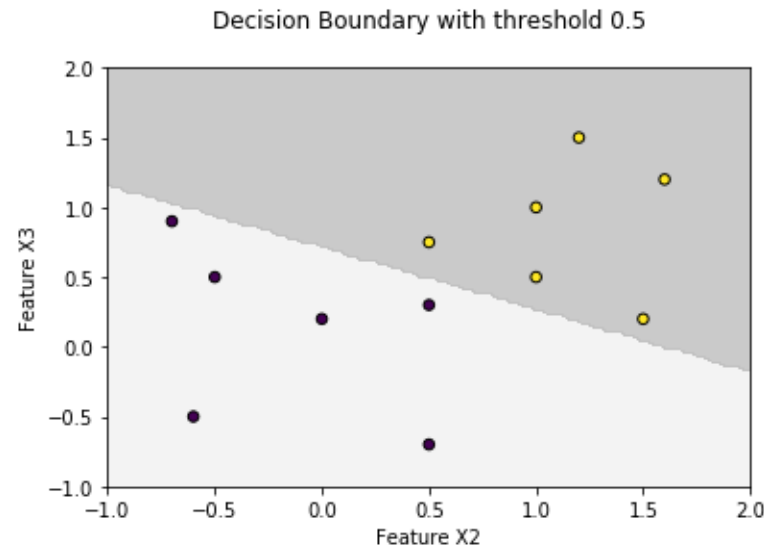
```
# Create meshgrid for decision boundary visualization
xx, yy = np.meshgrid(np.linspace(-1, 2, 100),
                     np.linspace(-1, 2, 100))
print('xx.shape = ', xx.shape)
```

```
xx.shape = (100, 100)
```

```
# Get output for the grid points to plot decision boundary
percept = copy.deepcopy(xx)
```

```
for ixx in range(100):
    for iyy in range(100):
        tmpout=W12*xx[ixx,iyy]+W13*yy[iyy]+theta
        if (tmpout > 0):
            percept[ixx,iyy] = 1
        else:
            percept[ixx,iyy] = 0
```

Partition the entire 2-D plane into 2 regions



```
# Visualize the decision boundary
```

```
threshold_1 = 0.5
```

```
plt.contourf(xx, yy, percept, levels=[0, threshold_1, 1], alpha=0.3, cmap=plt.cm.Greys)
plt.scatter(X2, X3, c=desiredout, cmap='viridis', edgecolors='k', s=30)
```

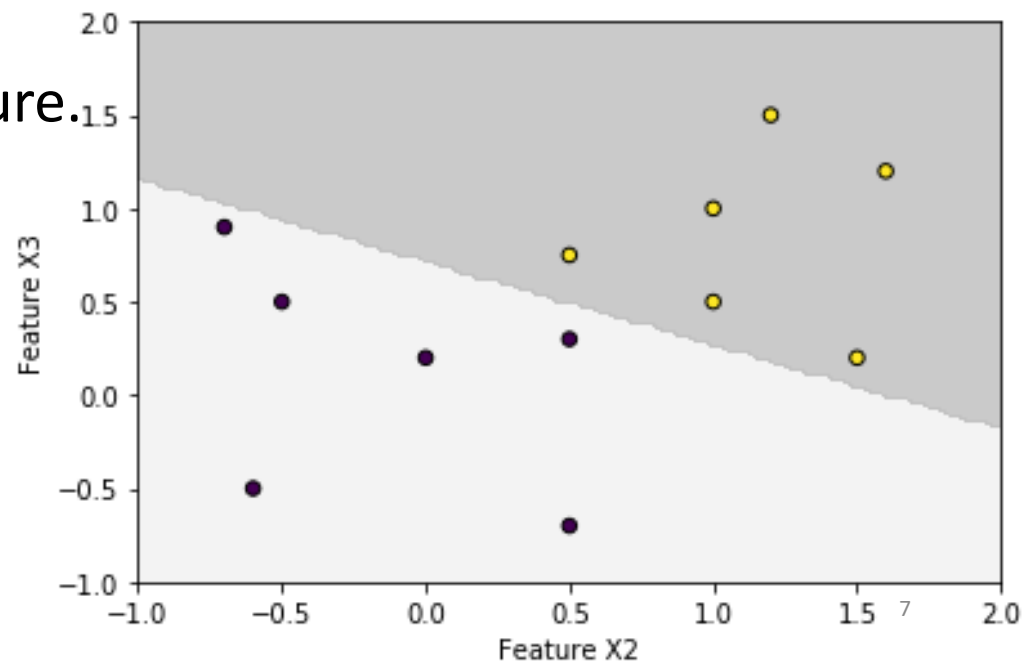
```
plt.title(f'Decision Boundary with threshold {threshold_1}\n')
plt.xlabel('Feature X2')
plt.ylabel('Feature X3')
```

```
plt.show()
```

End of demo of Part 1:

Your Tasks for Part 1:

1. Change the set of points to another set of **linearly separable** points (slide 3).
2. Re-run the training of the perceptron to get a new figure, with the successful training.
3. Note that changing the **threshold_1** value (shown on slide 6) would not change the figure. Do you know why ?



Use ANN / Keras(two I/P, one O/P):

For classification of the 12 data points into two classes

```
#  
# Part 2: ANN 12 data points linearly Separable  
#-----  
# Keras is demonstrated. It is a high-level API for building and training ANN  
# ANN model. The demo is for an ANN with two inputs, one output.  
# One hidden layer  
  
Number_hidden_units = 15  
  
def baseline_model1():  
    model = Sequential()  
    model.add(Dense(Number_hidden_units, activation='relu', input_dim = 2))  
  
    model.add(Dense(1, activation='sigmoid')) # Output layer for binary classification  
  
    model.compile(optimizer=SGD(learning_rate = 0.3), loss='binary_crossentropy',  
                  metrics=['accuracy'])  
  
    return model
```

Define the Keras model

```

# training instances are defined
tmpp= np.array([
    [1.0000 , 1.0000 , 1.0000],
    [1.0000 , 0.5000 , 1.0000],
    [1.5000 , 0.2000 , 1.0000],
    [0.5000 , 0.7500 , 1.0000],
    [1.2000 , 1.5000 , 1.0000],
    [1.6000 , 1.2000 , 1.0000],
    [0.5000 , 0.3000 , 0.0000],
    [0.5000 , -0.7000 , 0.0000],
    [0.0000 , 0.2 , 0.0000],
    [-0.7 , 0.9000 , 0.0000],
    [-0.6 , -0.5 , 0.0000],
    [-0.5 , 0.5 , 0.0000]
])
train_x=tmpp[:,0:2]
train_y=tmpp[:,2]

train_y = train_y.reshape((len(train_y),1))

print(train_x.shape)
print(train_y.shape)

ANN_model = baseline_model1()
ANN_model.summary()

```

Define the 12 data points,
and their class labels

Define the ANN_model
and show the summary

```

(12, 2)
(12, 1)

```

Model: "sequential_12"

Layer (type)	Output Shape	Param #
dense_26 (Dense)	(None, 15)	45
dense_27 (Dense)	(None, 1)	16

Total params: 61

Trainable params: 61

Non-trainable params: 0

Show the ANN_model
summary: **very useful**

```
start_time = time.time()

history = ANN_model.fit(train_x, train_y, epochs=100, verbose = 0)
# Use verbose = 1 for details of training

end_time = time.time()
exe_time = end_time - start_time
print("Execution time: ", exe_time)

scores = ANN_model.evaluate(train_x, train_y, verbose = 0)
print("Training Accuracy = ", scores)
```

**Training the ANN_model
using .fit**

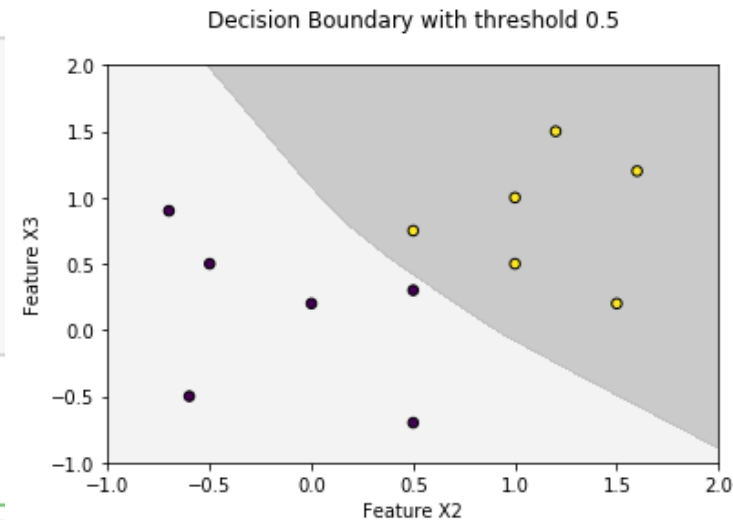
Execution time: 0.35161399841308594

Training Accuracy = [0.07894695550203323, 1.0]

```
xx, yy = np.meshgrid(np.linspace(-1, 2, 300),
                    np.linspace(-1, 2, 300))
grid = np.c_[xx.ravel(), yy.ravel()]
print('grid.shape is ', grid.shape)

print('xx.shape is ', xx.shape)
```

```
grid.shape is (90000, 2)
xx.shape is (300, 300)
```



```
ANN_probs = ANN_model.predict(grid).reshape(xx.shape)

# Visualize the decision boundary
threshold_1 = 0.5

X2=tmp[(:,0)]
X3=tmp[(:,1)]
desiredout=tmp[(:,2)]

plt.contourf(xx, yy, ANN_probs, levels=[0, threshold_1, 1], alpha=0.3,
            cmap=plt.cm.Greys)
plt.scatter(X2, X3, c=desiredout, cmap='viridis', edgecolors='k', s=30)

plt.title(f'Decision Boundary with threshold {threshold_1}\n')
plt.xlabel('Feature X2')
plt.ylabel('Feature X3')

plt.show()
```

.predict gives
the results with
given inputs

↑
The Figure

Show the entire grid using
the trained ANN_model

End of demo of Part 2:

Your Tasks for Part 2:

1. With reference to the figure of the previous slide 11, what do you notice about the boundary between the two classes of points ?
2. With reference to slide 10, why is that the number of parameters is 61 for this neural network ?

```

# Part 3: ANN with data points not linearly separable
#-----
# One hidden layer
# ANN demo: 20 training instances are defined
# These points are not linearly separable
#
tmpp= np.array([
    [1.0000 , 1.0000 , 1.0000],
    [1.1000 , 0.6000 , 1.0000],
    [1.5000 , 0.2000 , 1.0000],
    [-0.5000 , 0.7500 , 1.0000],
    [-0.5000 , 1.5000 , 1.0000],
    [-0.1000 , 0.9000 , 1.0000],
    [ 0.6000 , 1.2 , 1.0000],
    [1.5000 , 1.5 , 1.0000],
    [1.8000 , 1.0000 , 1.0000],
    [1.8000 , 0.2000 , 1.0000],
    [-0.8000 , -0.1000 , 0.0000],
    [-0.4000 , 0.2000 , 0.0000],
    [-0.6000 , 0.1500 , 0.0000],
    [-0.2000 , -0.4000 , 0.0000],
    [1.2000 , 0.1 , 0.0000],
    [1.5 , -0.6 , 0.0000],
    [0.5000 , 0.7500 , 0.0000],
    [0.5000 , 0.3000 , 0.0000],
    [0.5000 , -0.7000 , 0.0000],
    [0.0000 , 0.2000 , 0.0000],
])
print(tmpp.shape)

```

20 data points and their labels (two classes)

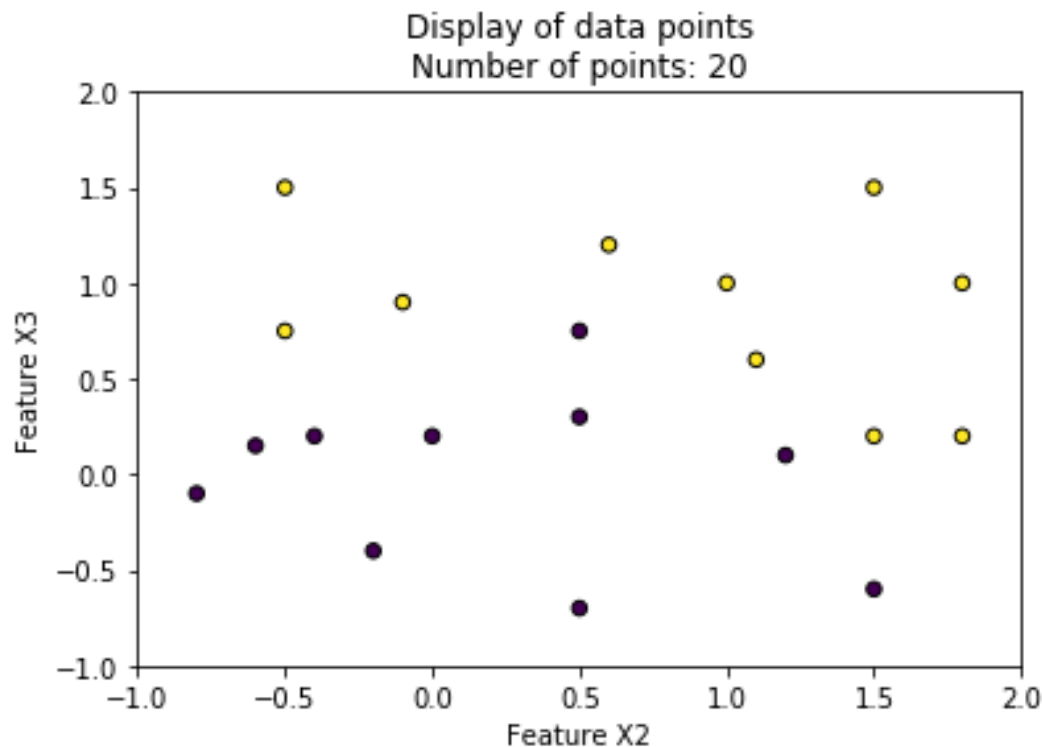
(20, 3)

```

X2=tmp[0]
X3=tmp[1]
desiredout=tmp[2]

Num_datapoints = len(X2)
plt.scatter(X2, X3, c=desiredout, cmap='viridis', edgecolors='k', s=30)
plt.title(f'Display of data points\n'
          f'Number of points: {Num_datapoints}')
plt.xlabel('Feature X2')
plt.ylabel('Feature X3')
plt.xlim(-1, 2) # xmin=-1, xmax=2
plt.ylim(-1, 2)
plt.show()

```



Display of the
20 data points
(two classes)

```
# ANN model of the two inputs
# One hidden layer
```

```
Number_hidden_units = 20
```

```
def baseline_model1():
    model = Sequential()
    model.add(Dense(Number_hidden_units, activation='relu', input_dim = 2))
    model.add(Dense(1, activation='sigmoid')) # Output layer for binary classification

    model.compile(optimizer=SGD(learning_rate = 0.3), loss='binary_crossentropy',
                    metrics=['accuracy'])

    return model
```

```
train_x=tmp[0:2]
train_y=tmp[2]
train_y = train_y.reshape((20,1))
print(train_x.shape)
print(train_y.shape)
```

```
ANN_model = baseline_model1()
ANN_model.summary()
```

```
(20, 2)
(20, 1)
Model: "sequential_13"
```

Layer (type)	Output Shape	Param #
dense_28 (Dense)	(None, 20)	60
dense_29 (Dense)	(None, 1)	21

Total params: 81
Trainable params: 81

Define Keras ANN model:
One hidden layer with 20 hidden units

Shows the model summary

```

start_time = time.time()

history = ANN_model.fit(train_x, train_y, epochs=400, verbose = 0)
# Use verbose = 1 for details of training

end_time = time.time()
exe_time = end_time - start_time
print("Execution time: ", exe_time)

scores = ANN_model.evaluate(train_x, train_y, verbose = 2)
print("Training Accuracy = ", scores)

```

Execution time: 0.6980419158935547

```

xx, yy = np.meshgrid(np.linspace(-1, 2, 300),
                    np.linspace(-1, 2, 300))
grid = np.c_[xx.ravel(), yy.ravel()]
print(grid.shape)

```

(90000, 2)

```
ANN_probs = ANN_model.predict(grid).reshape(xx.shape)
```

```

# Visualize the decision boundary
threshold_1 = 0.5

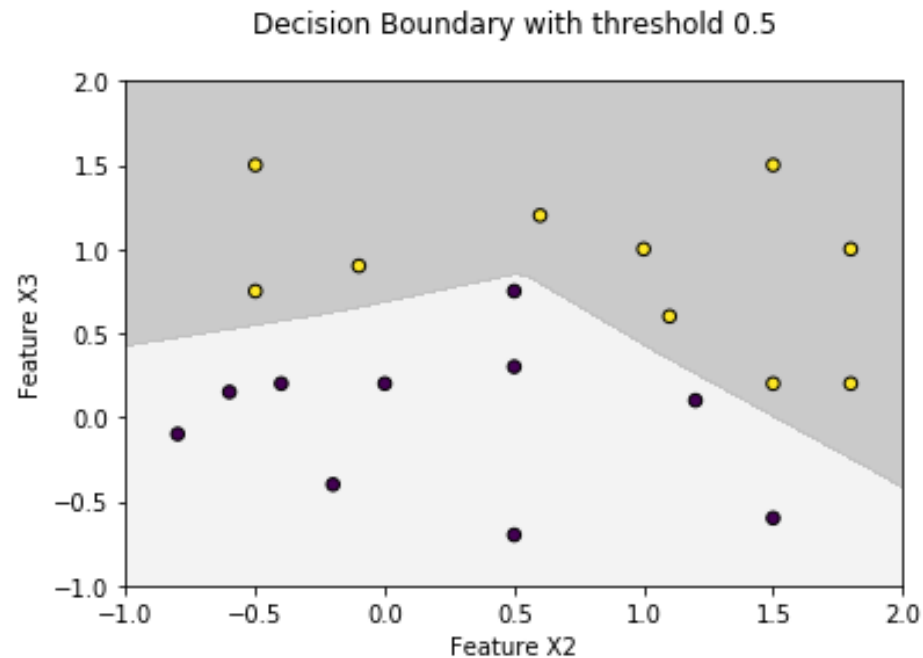
plt.contourf(xx, yy, ANN_probs, levels=[0, threshold_1, 1], alpha=0.3,
            cmap=plt.cm.Greys)
plt.scatter(X2, X3, c=desiredout, cmap='viridis', edgecolors='k', s=30)

plt.title(f'Decision Boundary with threshold {threshold_1}\n')
plt.xlabel('Feature X2')
plt.ylabel('Feature X3')

plt.show()

```

.fit is the training



**.predict gives
the results
with given
inputs**

End of demo of Part 3:

Your Tasks for Part 3:

1. With reference to slide 15, why is that the number of parameters is 81 for this neural network ?

```
# Part 4: ANN with data points not linearly separable
```

```
#-----
```

```
# ANN model of the two inputs, one output
```

```
# Two hidden layers
```

```
Number_hidden1_units = 15
```

```
Number_hidden2_units = 10
```

```
def baseline_model2():
```

```
    model = Sequential()
```

```
    model.add(Dense(Number_hidden1_units, activation='relu', input_dim = 2))
```

```
    model.add(Dense(Number_hidden2_units, activation='relu'))
```

```
    model.add(Dense(1, activation='sigmoid')) # Output layer for binary classification
```

```
    model.compile(optimizer=SGD(learning_rate = 0.15), loss='binary_crossentropy',  
                  metrics=['accuracy'])
```

```
    return model
```

```
ANN_model = baseline_model2()
```

```
ANN_model.summary()
```

Define Keras ANN model:

Two hidden layers with 15 / 10
hidden units

```
Model: "sequential_14"
```

Shows the model summary

Layer (type)	Output Shape	Param #
=====		
dense_30 (Dense)	(None, 15)	45
dense_31 (Dense)	(None, 10)	160
dense_32 (Dense)	(None, 1)	11
=====		

```
Total params: 216
```

```
Trainable params: 216
```

```
Non-trainable params: 0
```

```

# Start training

history = ANN_model.fit(train_x, train_y, epochs=300, verbose = 2)

ANN_probs = ANN_model.predict(grid).reshape(xx.shape)

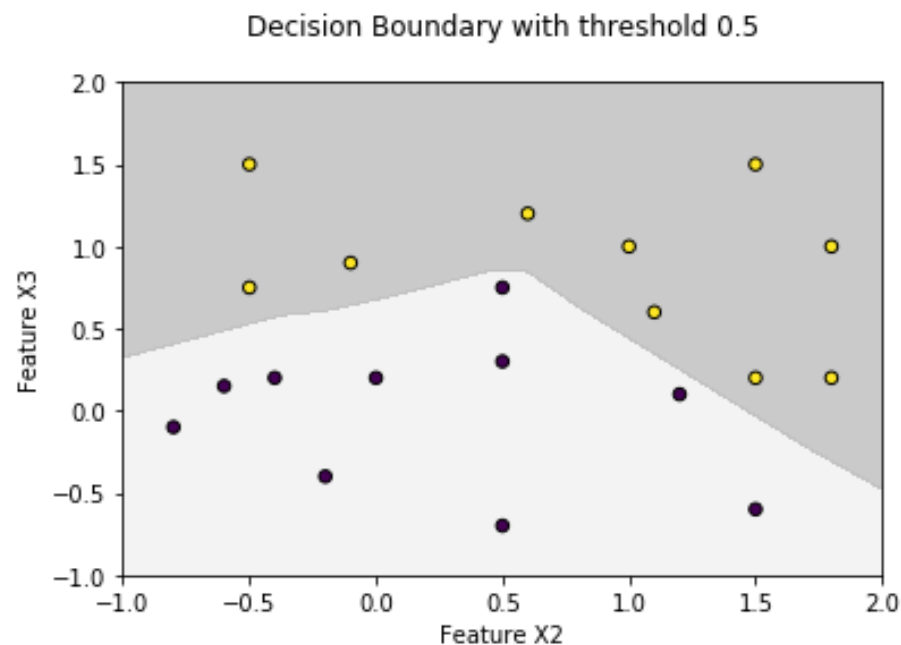
# Visualize the decision boundary
threshold_1 = 0.5

plt.contourf(xx, yy, ANN_probs, levels=[0, threshold_1, 1], alpha=0.3,
             cmap=plt.cm.Greys)
plt.scatter(X2, X3, c=desiredout, cmap='viridis', edgecolors='k', s=30)

plt.title(f'Decision Boundary with threshold {threshold_1}\n')
plt.xlabel('Feature X2')
plt.ylabel('Feature X3')

plt.show()

```



End of demo of Part 4:

Your Tasks for Part 4:

1. With reference to slide 18, why is that the number of parameters is 216 for this neural network ?
2. Change the **threshold_1** value (shown on slide 19) to 0.3, 0.4 , 0.6 or 0.7 to see a new figure.

```
# Part 5: ANN with data points not linearly separable
#-----
# The MLPClassifier is a framework in the sklearn.neural_network module
# of the Scikit-learn library. It implements MLP for supervised learning tasks,
# specifically for classification problems.
#
# ANN model of the two inputs, one output
# Three hidden layers is used.
```

MLPClassifier Model with 3 hidden layers

```
# Another ANN
# Define the hyperparameters for the neural network
hidden_layer_depth = 3 # Number of hidden layers
hidden_layer_width = 20 # Number of neurons per hidden layer

# Create a tuple representing the hidden layer sizes
hidden_layer_sizes = (hidden_layer_width,) * hidden_layer_depth
```

```
# Train a neural network on the training dataset
mlp = MLPClassifier(hidden_layer_sizes=hidden_layer_sizes,
                    activation='relu', # activation function for hidden layers
                    solver='sgd', # optimization algorithm
                    max_iter=2000)
mlp.fit(train_x, train_y)
```

solver = 'adam'
use adaptive learning rate

```
c:\users\gpang\appdata\local\programs\python\python36\lib\site-packages\sklearn\
ayer_perceptron.py:916: DataConversionWarning: A column-vector y was passed when
ted. Please change the shape of y to (n_samples, ), for example using ravel().
y = column_or_1d(y, warn=True)
```

```
MLPClassifier(activation='relu', alpha=0.0001, batch_size='auto', beta_1=0.9,
              beta_2=0.999, early_stopping=False, epsilon=1e-08,
              hidden_layer_sizes=(20, 20, 20), learning_rate='constant',
              learning_rate_init=0.001, max_iter=2000, momentum=0.9,
              n_iter_no_change=10, nesterovs_momentum=True, power_t=0.5,
              random_state=None, shuffle=True, solver='sgd', tol=0.0001,
              validation_fraction=0.1, verbose=False, warm_start=False)
```

```
print(train_x.shape)
print(train_y.shape)
print(xx.shape)
ANN_probs = mlp.predict(grid).reshape(xx.shape)
```

```
(20, 2)
(20, 1)
(300, 300)
```

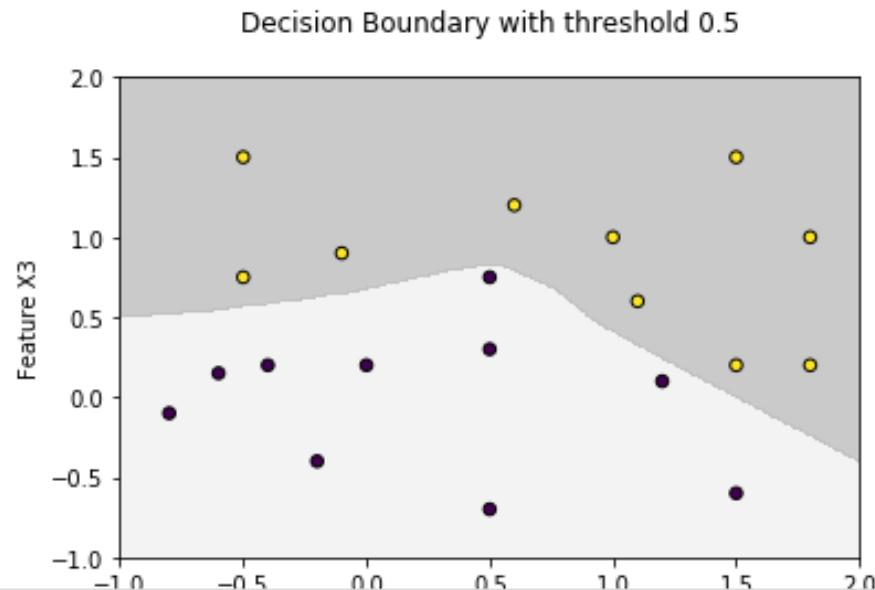
.predict gives the results

```
# Visualize the decision boundary
threshold_1 = 0.5

plt.contourf(xx, yy, ANN_probs, levels=[0, threshold_1, 1], alpha=0.3,
             cmap=plt.cm.Greys)
plt.scatter(X2, X3, c=desiredout, cmap='viridis', edgecolors='k', s=30)

plt.title(f'Decision Boundary with threshold {threshold_1}\n')
plt.xlabel('Feature X2')
plt.ylabel('Feature X3')

plt.show()
```



End of demo of Part 5:

Your Tasks for Part 5:

1. With reference to slide 21, change the `hidden_layer_width` to 25 (25 hidden units in each hidden layer). Re-run the training and classification code to generate a new figure.
2. Change the options in the use of `MLPClassifier`. Use the solver 'adam'. Modify the momentum parameter to another value to see if there is any change to the result.

```
: # Part 6: ANN with many data points not linearly separable
#-----
# The MLPClassifier is used again in this example.
```

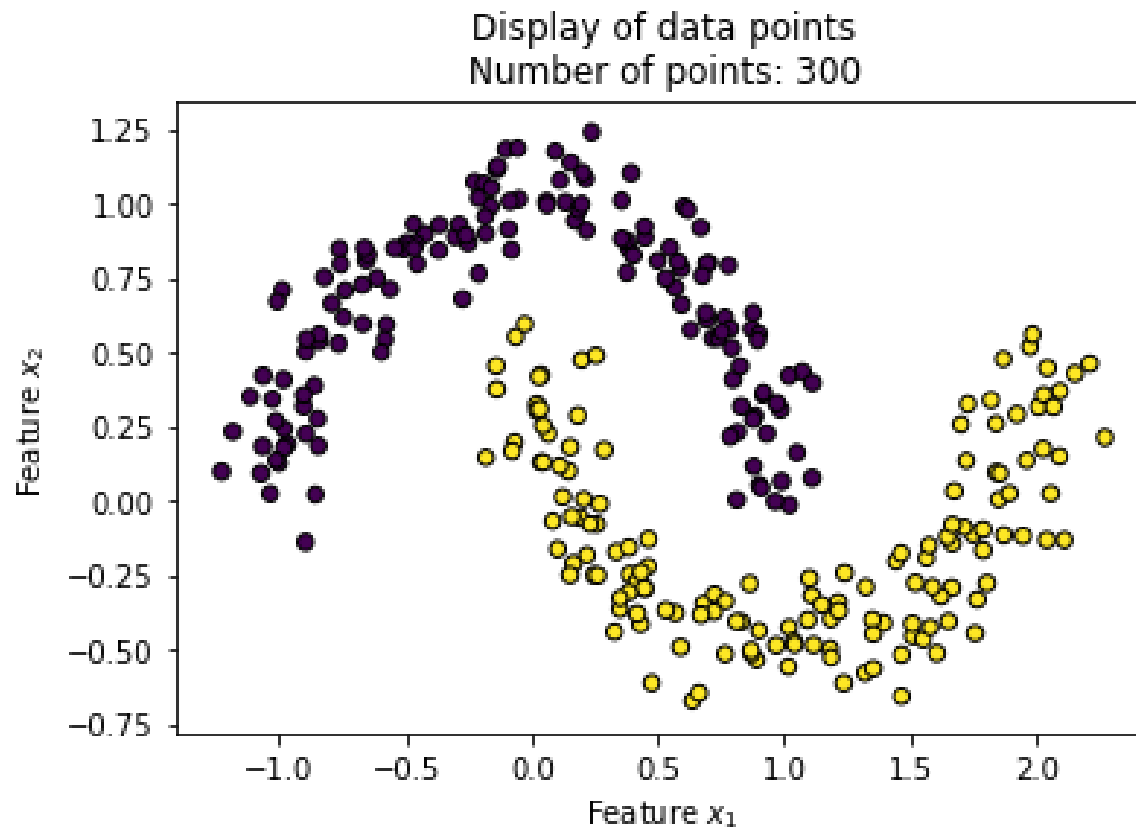
```
: # Create a simple dataset
Num_datapoints = 300
X, y = make_moons(n_samples=Num_datapoints, noise=0.1, random_state=42)
```

```
: print('X.shape: ', X.shape)
print('y.shape: ', y.shape)
print('min 1st column= ', np.min(X[:,0]), 'max 1st column= ', np.max(X[:,0]) )
print('min 2nd column= ', np.min(X[:,1]), 'max 2nd column= ', np.max(X[:,1]) )
print(X[0:3,:])
print(y)
```

```
X.shape: (300, 2)
y.shape: (300,)
min 1st column= -1.2272758742530834 max 1st column= 2.2703863872199213
min 2nd column= -0.6728418566502035 max 2nd column= 1.239950896359412
[[ 0.68298822 -0.34520334]
 [ 2.04099043 -0.13161467]
 [-0.13975154  0.4543905 ]]
[1 1 1 0 1 1 1 0 0 1 1 0 1 0 1 1 1 0 1 0 0 1 1 0 0 1 1 1 1 0 1 0 0 1 1 1 1 0 1 0 0 1 0 0
 0 0 0 0 0 1 0 0 0 1 1 0 1 1 0 1 0 1 1 0 0 1 0 1 0 0 1 1 0 0 1 0 1 0 0 1 1 1 0 0 1 0 1 0 0 1 1
 0 1 0 0 0 1 1 1 0 0 0 0 0 0 0 0 0 1 0 0 1 0 0 1 0 1 0 1 0 0 1 1 1 0 0 1 1 1 0 0 1 0 0
 1 0 1 1 0 0 1 1 1 0 1 1 1 0 1 0 0 1 1 0 0 1 0 1 1 1 1 1 0 0 0 1 0 1 1 0 0
 0 0 1 0 1 1 1 1 1 0 1 0 1 1 0 0 0 0 1 1 1 0 0 1 1 0 0 1 0 0 1 0 1 0 1 1 0
 1 1 1 0 0 0 1 1 1 1 0 1 1 0 1 0 0 0 1 1 1 1 0 0 0 0 1 0 0 1 0 0 1 1 0 0 1
 1 1 0 1 0 0 1 1 0 1 1 1 1 0 0 0 1 1 1 1 1 0 1 0 1 1 1 0 0 0 0 0 0 0 0 0 1
 0 0 0 1 0 0 1 1 0 1 1 0 0 1 1 1 1 1 0 0 1 1 0 1 1 1 1 0 0 1 0 0 1 0 1 0 1
 0 0 1 0]
```

```
: # Plotting of the dataset # cmap=plt.cm.Greys

plt.scatter(X[:, 0], X[:, 1], c=y, cmap='viridis', edgecolors='k', s=30)
plt.title(f'Display of data points\n'
          f'Number of points: {Num_datapoints}')
plt.xlabel('Feature $x_1$')
plt.ylabel('Feature $x_2$')
plt.show()
```



```
# Split the data into training and testing sets (80% train, 20% test)
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

# Standardize the features (important in machine learning)
#scaler = StandardScaler()
#X_train = scaler.fit_transform(X_train)
#X_test = scaler.transform(X_test)
```

Use of the train_test_split

```
# Define the hyperparameters for the neural network
hidden_layer_depth = 3 # Number of hidden layers
hidden_layer_width = 15 # Number of neurons per hidden layer

# Create a tuple representing the hidden layer sizes
hidden_layer_sizes = (hidden_layer_width,) * hidden_layer_depth

# Train a neural network on the training dataset
mlp = MLPClassifier(hidden_layer_sizes=hidden_layer_sizes,
                    activation='relu', # activation function for hidden layers
                    solver='sgd', # optimization algorithm
                    max_iter=3000)
mlp.fit(X_train, y_train)
```

ANN has 3
hidden layers,
each 15 hidden
nodes.

Training is using 240 points with labels

```
MLPClassifier(activation='relu', alpha=0.0001, batch_size='auto', beta_1=0.9,
              beta_2=0.999, early_stopping=False, epsilon=1e-08,
              hidden_layer_sizes=(15, 15, 15), learning_rate='constant',
              learning_rate_init=0.001, max_iter=3000, momentum=0.9,
              n_iter_no_change=10, nesterovs_momentum=True, power_t=0.5,
              random_state=None, shuffle=True, solver='sgd', tol=0.0001,
              validation_fraction=0.1, verbose=False, warm_start=False)
```

solver = 'adam'
use adaptive learning rate

```
# Make predictions and evaluate the testset accuracy
y_pred = mlp.predict(X_test)
accuracy = accuracy_score(y_test, y_pred)
print(f'Test Accuracy: {accuracy * 100:.2f}%')
```

Test Accuracy: 100.00%

.predict on the
data reserved
for testing

```
# Create meshgrid for decision boundary visualization
x_min, x_max = X_test[:, 0].min() - 0.5, X_test[:, 0].max() + 0.5
y_min, y_max = X_test[:, 1].min() - 0.5, X_test[:, 1].max() + 0.5
xx, yy = np.meshgrid(np.linspace(x_min, x_max, 300),
                    np.linspace(y_min, y_max, 300))
grid = np.c_[xx.ravel(), yy.ravel()]
print(grid.shape)
```

(90000, 2)

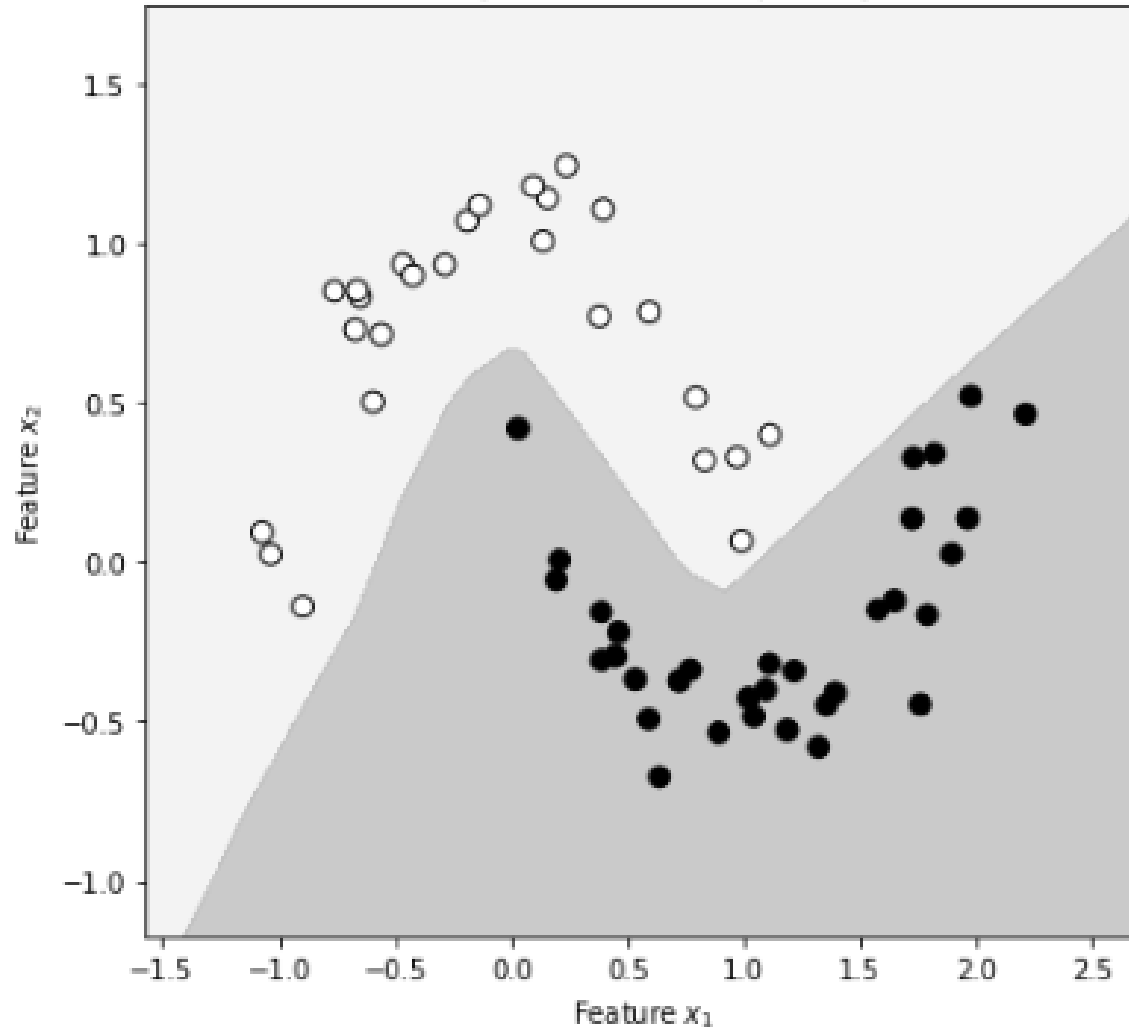
```
# Get probabilities for the grid points to plot decision boundaries
# Need to reshape to the same dimension as the xx
probs = mlp.predict_proba(grid[:, 1]).reshape(xx.shape)

fig, axes = plt.subplots(1, 2, figsize=(12, 6))

# Visualize the decision boundary
threshold_1 = 0.5
ax1 = axes[0]
ax1.contourf(xx, yy, probs, levels=[0, threshold_1, 1], alpha=0.3, cmap=plt.cm.Greys)
ax1.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap=plt.cm.Greys, edgecolors='k', s=60)
ax1.set_title(f'Decision Boundary with threshold {threshold_1}\n'
              f'Hidden Layers: {hidden_layer_depth}, Neurons per Layer: {hidden_layer_width}')
ax1.set_xlabel('Feature $x_1$')
ax1.set_ylabel('Feature $x_2$')
```

.predict to get
the probabilities;
show using
threshold of 0.5

Decision Boundary with threshold 0.5
Hidden Layers: 3, Neurons per Layer: 15



← The Figure showing the boundary and the 60 test points

```
print(hidden_layer_sizes)
```

```
(15, 15, 15)
```

```

# Part 7: ANN with 300 data points using Keras.
#-----
# ANN model of the two inputs, one output
# Two hidden layers

Number_hidden1_units = 15
Number_hidden2_units = 10

def baseline_model2():
    model = Sequential()
    model.add(Dense(Number_hidden1_units, activation='relu', input_dim = 2))
    model.add(Dense(Number_hidden2_units, activation='relu'))
    model.add(Dense(1, activation='sigmoid')) # Output layer for binary classification

    model.compile(optimizer=SGD(learning_rate = 0.15), loss='binary_crossentropy',
                  metrics=['accuracy'])

    return model

ANN_model = baseline_model2()
ANN_model.summary()

```

Model: "sequential_18"

Layer (type)	Output Shape	Param #
=====		
dense_42 (Dense)	(None, 15)	45
dense_43 (Dense)	(None, 10)	160
dense_44 (Dense)	(None, 1)	11
=====		

Total params: 216
Trainable params: 216
Non-trainable params: 0

Keras framework.
Why 216 parameters ?

```
y_train = y_train.reshape((240,1))
print(X_train.shape)
print(y_train.shape)
print(y_test.shape)
```

```
(240, 2)
(240, 1)
(60,)
```

```
# Start training
```

```
history = ANN_model.fit(X_train, y_train, epochs=300, verbose = 0)
```

```
ANN_probs = ANN_model.predict(grid).reshape(xx.shape)
```

```
# Visualize the decision boundary
```

```
threshold_1 = 0.5
```

```
plt.contourf(xx, yy, ANN_probs, levels=[0, threshold_1, 1], alpha=0.3,
             cmap=plt.cm.Greys)
plt.scatter(X_test[:, 0], X_test[:, 1], c=y_test, cmap=plt.cm.Greys,
           edgecolors='k', s=30)
```

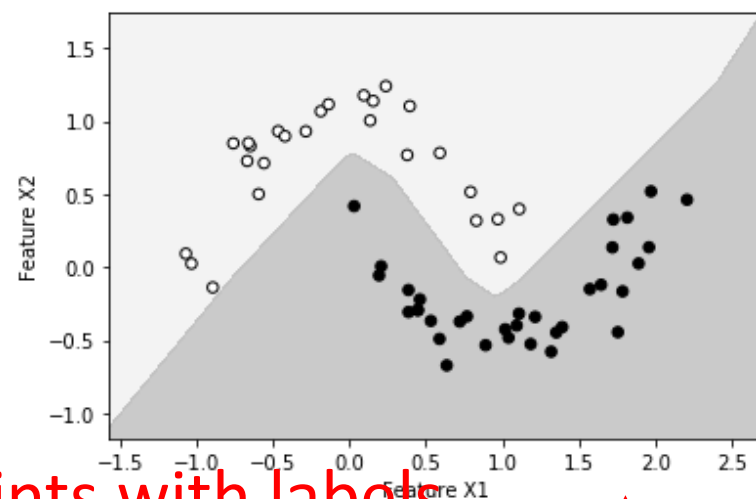
```
plt.title(f'Decision Boundary with threshold {threshold_1}\n')
```

```
plt.xlabel('Feature X1')
```

```
plt.ylabel('Feature X2')
```

```
plt.show()
```

Decision Boundary with threshold 0.5



Training is using 240 points with labels

The Figure
showing the
boundary
and the 60
test points

.predict to get the probabilities;
show using threshold of 0.5

Notes: Class02C_Assignment1_on_Classification.ipynb

1. Use the given jupyter file .ipynb as the basis for this assignment.
2. Submit your new .ipynb file in Moodle (online) by 11:45pm on the deadline date.

Your tasks only for Part 1 to 5.

3. Answer to question can be done by adding a new cell and write text into it:

